

Full Stack II Lab Manual – Detailed Step-by-Step Execution

Experiment 1: ExpressJS – Routing, HTTP Methods, Middleware

a. Route Handling and URL Parameters

Objective

Learn how routes, path parameters, and query strings work in ExpressJS.

Step-by-Step

1. Initialize a NodeJS Project

```
mkdir express-routing-demo
```

```
cd express-routing-demo
```

```
npm init -y
```

```
npm install express
```

2. Create app.js

```
// app.js
```

```
const express = require('express');
```

```
const app = express();
```

```
// Route with path parameter
```

```
app.get('/hello/:name', (req, res) => {  
  res.send(`Hello, ${req.params.name}!`);  
});
```

```
// Route with query parameter
```

```
app.get('/search', (req, res) => {  
  const term = req.query.term;
```

```
res.send(`Search term: ${term}`);  
});
```

```
app.listen(3000, () => console.log('Server running on port 3000'));
```

3. Run the Server

```
node app.js
```

4. Test the Endpoints

- Visit <http://localhost:3000/hello/John>
 - Output: Hello, John!
- Visit <http://localhost:3000/search?term=nodejs>
 - Output: Search term: nodejs

b. HTTP Methods: Accept, Retrieve, and Delete Data

Objective

Practice creating RESTful endpoints for basic CRUD operations.

Step-by-Step

1. Install Dependencies

```
npm install body-parser
```

2. Update/Create app.js

```
const express = require('express');  
const bodyParser = require('body-parser');  
const app = express();  
  
app.use(bodyParser.json());
```

```
let items = [];
```

```
// Add item (POST)
```

```
app.post('/item', (req, res) => {  
  items.push(req.body.name);  
  res.send('Item added');  
});
```

```
// Get all items (GET)
```

```
app.get('/items', (req, res) => {  
  res.json(items);  
});
```

```
// Delete item (DELETE)
```

```
app.delete('/item/:name', (req, res) => {  
  items = items.filter(i => i !== req.params.name);  
  res.send('Item deleted');  
});
```

```
app.listen(3000, () => console.log('Server running on port 3000'));
```

3. Test with Postman or cURL

- **Add an item:**

- POST to `http://localhost:3000/item` with JSON body: `{ "name": "Book" }`
- Response: Item added

- **Get all items:**

- GET `http://localhost:3000/items`

- Response: ["Book"]
 - **Delete an item:**
 - DELETE http://localhost:3000/item/Book
 - Response: Item deleted
-

c. Middleware Demonstration

Objective

Understand middleware and how it intercepts requests.

Step-by-Step

1. Update/Create app.js

```
const express = require('express');
const app = express();

// Logging middleware
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next(); // Pass control to the next handler
});

app.get('/', (req, res) => res.send('Middleware demo'));

app.listen(3000, () => console.log('Server running on port 3000'));
```

2. Run and Observe

- Every time you access a route, e.g., http://localhost:3000/, the console logs:

GET /

Experiment 2: ExpressJS – Templating, Form Data

a. Using Templating Engine (EJS)

Objective

Render dynamic HTML using EJS templates.

Step-by-Step

1. Install Dependencies

```
npm install express ejs
```

2. Project Structure

```
/views
```

```
  profile.ejs
```

```
app.js
```

3. Create EJS Template

- views/profile.ejs

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
  <h1>User Profile</h1>
```

```
  <p>Name: <%= user.name %></p>
```

```
  <p>Age: <%= user.age %></p>
```

```
</body>
```

```
</html>
```

4. App Code

```
const express = require('express');

const app = express();

app.set('view engine', 'ejs');

app.get('/', (req, res) => {
  const user = { name: "Alice", age: 22 };
  res.render('profile', { user });
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

5. Run and Test

- Visit <http://localhost:3000/> to see the rendered HTML.

b. Handling Form Data

Objective

Accept user input from a form and display it.

Step-by-Step

1. Install body-parser

```
npm install body-parser
```

2. Project Structure

```
/views
```

```
  form.ejs
```

```
  result.ejs
```

```
app.js
```

3. Create Templates

- views/form.ejs

```
<form action="/submit" method="POST">  
  Name: <input type="text" name="name">  
  <button type="submit">Submit</button>  
</form>
```

- views/result.ejs

```
<h2>Hello, <%= name %>!</h2>
```

4. App Code

```
const express = require('express');  
const bodyParser = require('body-parser');  
const app = express();  
  
app.set('view engine', 'ejs');  
app.use(bodyParser.urlencoded({ extended: true }));  
  
app.get('/', (req, res) => res.render('form'));  
  
app.post('/submit', (req, res) => {  
  res.render('result', { name: req.body.name });  
});  
  
app.listen(3000, () => console.log('Server running on port 3000'));
```

5. Run and Test

- Fill the form and see the greeting on submission.

Experiment 3: ExpressJS – Cookies, Sessions, Authentication

a. Session Management Using Cookies and Sessions

Objective

Manage user sessions and cookies.

Step-by-Step

1. Install express-session

```
npm install express express-session
```

2. App Code

```
const express = require('express');  
const session = require('express-session');  
const app = express();
```

```
app.use(session({  
  secret: 'secretKey',  
  resave: false,  
  saveUninitialized: true,  
  cookie: { maxAge: 60000 }  
}));
```

```
app.get('/login', (req, res) => {  
  req.session.username = 'testuser';  
  res.send('User logged in');  
});
```



```
app.get('/dashboard', (req, res) => {  
  if (req.session.username) {  
    res.send(`Welcome ${req.session.username}`);  
  } else {  
    res.send('Please log in first.');
```

```
  }  
});  
  
app.get('/logout', (req, res) => {  
  req.session.destroy();  
  res.send('Logged out');
```

```
});  
  
app.listen(3000, () => console.log('Server running on port 3000'));
```

3. Test

- Visit /login → /dashboard → /logout.

b. User Authentication

Objective

Authenticate users and manage login sessions.

Step-by-Step

1. Install Dependencies

```
npm install express body-parser express-session
```

2. App Code

```
const express = require('express');

const bodyParser = require('body-parser');

const session = require('express-session');

const app = express();


app.use(bodyParser.urlencoded({ extended: true }));

app.use(session({
  secret: 'authSecret',
  resave: false,
  saveUninitialized: true
}));


const users = { 'admin': '1234' };


app.get('/login', (req, res) => {
  res.send('<form method="post"><input name="username"/><input name="password" type="password"/><button>Login</button></form>');
});


app.post('/login', (req, res) => {
  const { username, password } = req.body;
  if (users[username] && users[username] === password) {
    req.session.user = username;
    res.send('Login successful');
  } else {
    res.send('Invalid credentials');
  }
}
```

```
});
```

```
app.get('/home', (req, res) => {  
  if (req.session.user) {  
    res.send(`Welcome, ${req.session.user}`);  
  } else {  
    res.send('Not authenticated');  
  }  
});
```

```
app.listen(3000, () => console.log('Server running on port 3000'));
```

3. Test

- Visit /login, submit form with username: admin, password: 1234.
- Visit /home to verify login.

Experiment 4: ExpressJS – Database, RESTful APIs

a. Connect to MongoDB Using Mongoose and Perform CRUD

Objective

Connect ExpressJS to MongoDB and perform CRUD operations.

Step-by-Step

1. Install Dependencies

```
npm install express mongoose body-parser
```

2. Start MongoDB

- Use local MongoDB or [MongoDB Atlas](#).

3. App Code

```
const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
const app = express();

app.use(bodyParser.json());

mongoose.connect('mongodb://localhost:27017/express_crud', {
  useNewUrlParser: true,
  useUnifiedTopology: true
});

const Item = mongoose.model('Item', { name: String });

// Create
app.post('/items', async (req, res) => {
  const item = new Item({ name: req.body.name });
  await item.save();
  res.send('Item saved');
});

// Read
app.get('/items', async (req, res) => {
  const items = await Item.find();
  res.json(items);
});
```

```
// Update
app.put('/items/:id', async (req, res) => {
  await Item.findByIdAndUpdate(req.params.id, { name: req.body.name });
  res.send('Item updated');
});

// Delete
app.delete('/items/:id', async (req, res) => {
  await Item.findByIdAndDelete(req.params.id);
  res.send('Item deleted');
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

4. Test with Postman

- POST, GET, PUT, DELETE as per REST conventions.

b. Build a Single Page Application Using RESTful APIs

Objective

Create a frontend that interacts with the backend REST API.

Step-by-Step

1. Ensure Backend is Running
2. Create index.html

```
<!DOCTYPE html>

<html>

<body>

  <h2>Items</h2>
```

```
<input id="itemInput" placeholder="Enter item">
<button onclick="addItem()">Add</button>
<ul id="itemList"></ul>
<script>
  async function fetchItems() {
    const res = await fetch('/items');
    const items = await res.json();

    document.getElementById('itemList').innerHTML =
      items.map(i => `<li>${i.name} <button
onclick="deleteItem('${i._id}')">Delete</button></li>`).join("");
  }
  async function addItem() {
    const name = document.getElementById('itemInput').value;
    await fetch('/items', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ name })
    });
    fetchItems();
  }
  async function deleteItem(id) {
    await fetch(`/items/${id}`, { method: 'DELETE' });
    fetchItems();
  }
  fetchItems();
</script>
</body>
```

</html>

3. Serve HTML (Optional)

- Add to Express:

```
app.use(express.static(__dirname));
```

- Open index.html in browser.

Experiments 5–9: ReactJS (Detailed for Each Subtask)

a. Render HTML to a Web Page

Objective

Render static HTML using React.

Step-by-Step

1. Create React App

```
npx create-react-app myapp
```

```
cd myapp
```

2. Edit src/App.js

```
function App() {  
  return (  
    <div>  
      <h1>Welcome to React</h1>  
      <p>This is rendered using JSX.</p>  
    </div>  
  );  
}  
  
export default App;
```

3. Run App

npm start

b. Writing Markup with JSX

1. Create Greeting.js

```
function Greeting() {  
  const name = 'Alice';  
  return (  
    <div>  
      <h2>Hello, {name}!</h2>  
    </div>  
  );  
}  
export default Greeting;
```

2. Import and Use in App.js

```
import Greeting from './Greeting';  
function App() {  
  return (  
    <div>  
      <Greeting />  
    </div>  
  );  
}  
export default App;
```

c. Creating and Nesting Components

1. Create Header.js

```
function Header() {  
  return <h1>This is a Header</h1>;  
}  
  
export default Header;
```

2. Use in Class Component

```
import React from 'react';  
import Header from './Header';  
  
class App extends React.Component {  
  render() {  
    return (  
      <div>  
        <Header />  
        <p>This is a class-based component</p>  
      </div>  
    );  
  }  
}  
  
export default App;
```

d. Working with Props and State

1. ParentComponent.js

```
import React from 'react';
import Child from './Child';
function ParentComponent() {
  return <Child name="Alice" />;
}
export default ParentComponent;
```

2. Child.js

```
import React, { useState } from 'react';
function Child({ name }) {
  const [age, setAge] = useState(20);
  return (
    <div>
      <h1>Hello, {name}</h1>
      <p>Age: {age}</p>
      <button onClick={() => setAge(age + 1)}>Increase Age</button>
    </div>
  );
}
export default Child;
```

e. Add Styles (CSS & SASS Styling) and Display Data

1. Create styles.css

```
.title {
  color: blue;
```

```
font-size: 24px;
margin-bottom: 10px;
}
```

2. StyledComponent.js

```
import React from 'react';
import './styles.css';
function StyledComponent() {
  const inlineStyle = { color: 'green', fontWeight: 'bold' };
  return (
    <div>
      <h1 className="title">Styled using CSS</h1>
      <p style={inlineStyle}>This paragraph uses inline style</p>
    </div>
  );
}
export default StyledComponent;
```

f. Responding to Events

1. EventDemo.js

```
import React, { useState } from 'react';
function EventDemo() {
  const [message, setMessage] = useState('Hello!');
  const handleClick = () => setMessage('Button Clicked!');
  return (
    <div>
```

```
    <p>{message}</p>
    <button onClick={handleClick}>Click Me</button>
  </div>
);
}
export default EventDemo;
```

g. Conditional Rendering

1. ConditionalComponent.js

```
import React, { useState } from 'react';
function ConditionalComponent() {
  const [isLoggedIn, setIsLoggedIn] = useState(false);
  return (
    <div>
      {isLoggedIn ? <h2>Welcome Back!</h2> : <h2>Please Log In</h2>}
      <button onClick={() => setIsLoggedIn(!isLoggedIn)}>
        {isLoggedIn ? 'Logout' : 'Login'}
      </button>
    </div>
  );
}
export default ConditionalComponent;
```

h. Rendering Lists

1. ListComponent.js

```
import React from 'react';

function ListComponent() {

  const fruits = ['Apple', 'Banana', 'Cherry'];

  return (

    <ul>

      {fruits.map((fruit, index) => (

        <li key={index}>{fruit}</li>

      ))}

    </ul>

  );

}

export default ListComponent;
```

i. React Forms

1. FormComponent.js

```
import React, { useState } from 'react';

function FormComponent() {

  const [name, setName] = useState("");

  const [submittedName, setSubmittedName] = useState("");

  const handleSubmit = (e) => {

    e.preventDefault();

    setSubmittedName(name);

  };

  return (

    <div>

      <form onSubmit={handleSubmit}>
```

```

    <label>Name: </label>

    <input value={name} onChange={(e) => setName(e.target.value)} />

    <button type="submit">Submit</button>

  </form>

  {submittedName && <h2>Hello, {submittedName}</h2>}

</div>

);
}

export default FormComponent;

```

j. React Router

1. Install Router

```
npm install react-router-dom
```

2. App.js

```

import React from 'react';
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function Home() { return <h2>Home Page</h2>; }
function About() { return <h2>About Page</h2>; }

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link> | <Link to="/about">About</Link>

```

```
    </nav>

    <Routes>

      <Route path="/" element={<Home />} />

      <Route path="/about" element={<About />} />

    </Routes>

  </BrowserRouter>

);
}

export default App;
```

k. Updating the Screen

1. Counter.js

```
import React, { useState } from 'react';

function Counter() {

  const [count, setCount] = useState(0);

  return (

    <div>

      <p>Count: {count}</p>

      <button onClick={() => setCount(count + 1)}>Increment</button>

    </div>

  );

}

export default Counter;
```

l. Using Hooks

1. HookDemo.js

```

import React, { useState, useEffect } from 'react';

function HookDemo() {
  const [count, setCount] = useState(0);
  useEffect(() => {
    console.log("Count updated:", count);
  }, [count]);
  return (
    <div>
      <button onClick={() => setCount(count + 1)}>Click {count}</button>
    </div>
  );
}

export default HookDemo;

```

m. Sharing Data Between Components

1. ParentComponent.js

```

import React, { useState } from 'react';
import Child from './Child';

function ParentComponent() {
  const [message, setMessage] = useState("Initial Message");
  const updateMessage = () => setMessage("Updated from Child");
  return (
    <div>
      <h2>{message}</h2>
      <Child updateMessage={updateMessage} />
    </div>
  );
}

```



```
</div>

);
}

export default ParentComponent;
```

2. Child.js

```
import React from 'react';

function Child({ updateMessage }) {

  return <button onClick={updateMessage}>Update Parent</button>;

}

export default Child;
```

Experiment 10: MongoDB – Installation, Configuration, CRUD Operations

a. Install MongoDB and Configure Atlas

1. Go to [MongoDB Atlas](#)
2. Sign Up and Create a Cluster
3. Create Database User and Whitelist IP
4. Get Connection String

b. CRUD Operations Using insert(), find(), update(), remove()

1. Install MongoDB Node Driver

```
npm install mongodb
```

2. App Code

```
const { MongoClient } = require('mongodb');

const uri = 'your-mongodb-uri';
```

```
const client = new MongoClient(uri);

async function run() {
  try {
    await client.connect();
    const db = client.db('labdb');
    const collection = db.collection('students');
    await collection.insertOne({ name: 'John', age: 22 });
    const docs = await collection.find().toArray();
    console.log('Find:', docs);
    await collection.updateOne({ name: 'John' }, { $set: { age: 23 } });
    await collection.deleteOne({ name: 'John' });
  } finally {
    await client.close();
  }
}

run();
```

Experiment 11: MongoDB – Databases, Collections and Records

a. Create and Drop Databases and Collections

1. MongoDB Shell Commands

```
mongo
use mydb
db.createCollection("products")
db.products.drop()
db.dropDatabase()
```

b. Work with Records using find(), limit(), sort(), createIndex(), aggregate()

1. Insert Sample Data

```
db.products.insertMany([
  { name: 'Pen', price: 10 },
  { name: 'Pencil', price: 5 },
  { name: 'Eraser', price: 7 }
])
```

2. Queries

- Find all: `db.products.find()`
- Limit: `db.products.find().limit(2)`
- Sort: `db.products.find().sort({ price: -1 })`
- Create index: `db.products.createIndex({ name: 1 })`
- Aggregate:

```
db.products.aggregate([
  { $group: { _id: null, total: { $sum: "$price" } } }
])
```

Experiment 12: Augmented Programs

a. To-Do List Application using NodeJS and ExpressJS

1. Project Structure

```
/views
  index.ejs
app.js
```

2. App Code

```
const express = require("express");
const bodyParser = require("body-parser");
const app = express();

app.set("view engine", "ejs");
app.use(bodyParser.urlencoded({ extended: true }));

let tasks = [];

app.get("/", (req, res) => {
  res.render("index", { tasks });
});

app.post("/add", (req, res) => {
  const task = req.body.task;
  if (task) tasks.push(task);
  res.redirect("/");
});

app.post("/delete", (req, res) => {
  const taskToDelete = req.body.task;
  tasks = tasks.filter(t => t !== taskToDelete);
  res.redirect("/");
});
```

```
app.listen(3000, () => console.log("Server running on port 3000"));
```

3. EJS Template

- views/index.ejs

```
<!DOCTYPE html>

<html>

<body>

  <h2>To-Do List</h2>

  <form action="/add" method="post">

    <input name="task" required />

    <button type="submit">Add</button>

  </form>

  <ul>

    <% tasks.forEach(task => { %>

      <li>

        <form action="/delete" method="post" style="display:inline">

          <%= task %>

          <input type="hidden" name="task" value="<%= task %>" />

          <button type="submit">Delete</button>

        </form>

      </li>

    <% }) %>

  </ul>

</body>

</html>
```

b. Quiz App using ReactJS

1. Create React App

```
npx create-react-app quiz-app
```

```
cd quiz-app
```

2. Quiz.js

```
import React, { useState } from 'react';
```

```
const questions = [  
  { q: "Capital of India?", options: ["Delhi", "Mumbai", "Kolkata"], answer: "Delhi" },  
  { q: "2 + 2?", options: ["3", "4", "5"], answer: "4" }  
];
```

```
function Quiz() {  
  const [index, setIndex] = useState(0);  
  const [score, setScore] = useState(0);  
  const [showResult, setShowResult] = useState(false);  
  
  const handleOption = (selected) => {  
    if (selected === questions[index].answer) setScore(score + 1);  
    const next = index + 1;  
    if (next < questions.length) setIndex(next);  
    else setShowResult(true);  
  };  
  
  return (  
    <div>
```

```

    {!showResult ? (
      <div>
        <h3>{questions[index].q}</h3>
        {questions[index].options.map((opt, i) => (
          <button key={i} onClick={() => handleOption(opt)}>{opt}</button>
        ))}
      </div>
    ) : (
      <h2>Your Score: {score}/{questions.length}</h2>
    )}
  </div>
);
}
export default Quiz;

```

3. Use in App.js

```

import Quiz from './Quiz';

function App() {
  return <Quiz />;
}

export default App;

```

4. Run

```
npm start
```

General Tips

- Always start with `npm init -y` and install required packages.

- **Organize your files and folders as per best practices.**
- **Test your endpoints/components after each step.**
- **Use Postman/cURL for backend and browser for frontend.**
- **For MongoDB, ensure your database server is running or Atlas is configured.**